Advance Database System

Course Project Report

Medical Voice Memo & Patient Record Database System

(Relivox Webpage)

Submitted by: Disha Bharatbhai Patel

ABSTRACT

The Relivox Medical Voice Memo & Patient Record Database System is designed to streamline the clinical documentation process by integrating audio recordings, automated transcription, and structured medical reporting within a centralized relational database. The project focuses on building a secure, efficient, and scalable data architecture that supports the entire life cycle of clinical voice memo, beginning with the creation of a recording, transitioning through transcription, and culminating in the generation and storage of medical reports.

Through systematic requirement analysis, conceptual modeling, logical schema development, and a fully implemented physical design in MySQL, the Relivox system demonstrates the end-to-end workflow of a modern healthcare documentation application. The project incorporates essential database concepts such as normalization, referential integrity, indexing, and schema optimization, along with advanced features including stored procedures, triggers, views, and functions to ensure data integrity, auditability, and enhanced performance.

A user-friendly frontend built with PHP provides clinicians and administrators with the ability to create, access, and manage patient information, recordings, and reports in real time. The system is deployed on Rowan University's Elvis server, enabling seamless backend–frontend integration and real-world accessibility.

Overall, the Relivox project highlights the practical application of advanced database design principles within a healthcare context. It underscores the importance of structured data management in improving documentation accuracy, enhancing workflow efficiency, and ensuring secure access to sensitive medical information. The completed system serves as a robust foundation that can be expanded into a full-scale clinical documentation platform in future iterations.

Table of Contents

ABSTRACT 2
1. REQUIREMENTS COLLECTION & ANALYSIS 4
1.1 Problem Statement 4
1.2 Research Summary 4
1.3 System Design: 5
1.4 System Requirements: 6
1.4.1 Functional Requirements (Database-Centric) 6
1.4.2 Transaction Frequency: 6
1.4.3 Flow of Information: 7
2. CONCEPTUAL & LOGICAL DESIGN: 10
2.1 ER Diagram: 10
2.2 Normalization: 11
2.2.1 First Normal Form (1NF) 11
2.2.2 Second Normal Form (2NF) 11
2.2.3 Third Normal Form (3NF) 12
2.3 EER Diagram 12
3. PHYSICAL DESIGN & PERFORMANCE 14
4. FRONTEND DESIGN & IMPLEMENTATION 15
5. CONCLUSION: 18

1. REQUIREMENTS COLLECTION & ANALYSIS

1.1 Problem Statement

Healthcare professionals frequently rely on voice dictation to capture patient information, clinical observations, and progress notes during busy workflows. However, many medical practices lack a centralized and efficient system to manage these voice memos, their transcriptions, and the resulting clinical reports. As a result, important data becomes scattered across devices, stored in inconsistent formats, or manually transferred between applications. This leads to delays, transcription errors, fragmented patient records, and reduced documentation efficiency.

Additionally, clinicians need a reliable way to retrieve previous recordings, verify transcription accuracy, and maintain compliance with healthcare documentation standards. Without an integrated database-driven solution, managing patient information becomes time-consuming and error-prone. Medical organizations also require secure audit logging, user access control, and structured data storage to ensure the integrity and confidentiality of sensitive patient records.

The Relivox system aims to address these challenges by providing a unified platform that connects voice recordings, automated transcriptions, and structured medical reports within a secure and well-designed relational database. Through proper data modeling, normalization, and backend–frontend integration, the system ensures that clinicians can efficiently create, manage, and retrieve clinical documentation while maintaining data accuracy, accessibility, and security.

1.2 Research Summary

The Relivox Medical Voice Memo & Patient Record Database System was created to provide a unified, efficient, and secure platform for managing the complete workflow of clinical documentation. In many healthcare environments, clinicians rely on voice dictation to capture patient information, yet the absence of an integrated system often results in scattered recordings, inconsistent transcription processes, and fragmented patient records. This project addresses these challenges by designing a robust relational database capable of linking patients, users, recordings, transcriptions, and medical reports in a seamless, structured manner. Beginning with comprehensive requirement analysis, the system identifies critical data needs and models them through a detailed conceptual and logical design that ensures referential integrity, normalization, and reduced redundancy throughout the database.

Building on this foundation, the project implements a fully functional physical database in MySQL, enhanced with views, functions, stored procedures, and triggers to support automation, auditing, and optimized data retrieval. A PHP-based frontend interface enables clinicians and administrators to perform real-time CRUD operations, manage patient and report data, and

monitor system activity through an intuitive web interface. The application is deployed on Rowan University's Elvis server, demonstrating a fully integrated backend–frontend system capable of supporting real-world clinical workflows. Together, these components showcase how thoughtful database design and implementation can improve documentation accuracy, streamline medical processes, and provide a scalable solution for future enhancements in healthcare technology.

1.3 System Design:

Medical data requires strict referential integrity (patient ↔ record ↔ transcription).
Databases must support audit logging, role-based access control, and transaction reliability.
Systems like Epic EHR and Cerner use relational models (SQL-based) with FHIR (Fast Healthcare Interoperability Resources) structure similar to what Relivox will follow.
The backend must allow fast read/write transactions for frequent insertions (recordings, notes, logs) and updates.

Actor	**Role**	**Interaction with the Database**
Doctor / Clinician	Main user	Creates new recordings, views and updates transcriptions, saves reports, retrieves past patient data
Patient	Data entity (not active user)	Patient details are stored and linked to reports and recordings
Administrator	System manager	Manages user accounts, access control, data backups, and logs
System / Application Layer	Interface layer	Communicates between the mobile app frontend and the database (API requests, inserts, updates)

Target Users: Licensed medical practitioners and administrators
Geographic Scope: U.S. based healthcare practices (HIPAA-compliant), scalable to global healthcare systems.

1.4 System Requirements:

1.4.1 Functional Requirements (Database-Centric)

- The system must store patient demographic details (name, contact, DOB, gender, address).
- Each voice recording must have a unique ID and metadata (duration, timestamp, file path).
- The database should store transcription text linked to a specific recording.
- Each transcription can generate one or more formatted reports (Progress or Nurse Note).
- Users (doctors/admins) must be stored with role identifiers and login credentials.
- Maintain foreign key relationships among tables (PatientID, RecordingID, ReportID).
- Enforce unique constraints on IDs and medical record numbers.
- Provide query optimization for quick retrieval (e.g., fetch all reports for a patient).
- Support CRUD operations for all major entities.
- Implement referential integrity between related entities (recording → transcription → report).
- Maintain transaction consistency during record insertions (atomic updates).
- Store timestamps for creation and modification of each entry.
- Implement audit logs for changes to patients or report data.
- Support user access levels doctors can read/write, admins can modify or delete.
- Enable search by patient name, date, or note type.
- Store error or failure logs for system auditing.
- Maintain data backups at regular intervals.
- Provide soft delete (inactive status) for records instead of full deletion.
- Allow integration with external APIs (e.g., for transcription storage).
- Enforce data encryption for sensitive patients and recording data.
- Support auto-incrementing primary keys for record tracking
- Handle concurrent transactions safely without data loss.
- Include validation constraints (non-null, correct format, date ranges).
- Allow summary reports or statistical queries (e.g., number of patients seen per week).
- Include indexing on key columns (PatientID, RecordingDate) for performance.

1.4.2 Transaction Frequency:

The Relivox system is designed to support a variety of database transactions that occur at different frequencies depending on user activity and operational needs. High-frequency transactions include the frequent insertion of new data such as patient encounters, voice recordings, automated transcriptions, and generated reports actions typically performed multiple times throughout a clinician's workday. Medium-frequency transactions involve updates to existing records, such as editing report content, adjusting patient information, modifying transcription accuracy details, or changing the status of reports and attachments. Low-frequency

transactions consist of operations like account management by administrators, audit log reviews, data archival, and database backups, which occur periodically but remain essential for long-term system integrity and compliance. Together, these transaction patterns ensure that the system remains responsive, scalable, and reliable while handling continuous clinical documentation activity.

1.4.3 Flow of Information:

After analyzing the system requirements, the following entities, relationships, and constraints were identified. The goal is to define how data will be logically stored, linked, and constrained to ensure consistency and efficiency in the Relivox Medical Voice Memo Database.

1. Entity Types and Their Attributes

1. User: Stores information about each system user (doctor, admin).
Attributes:
UserID, FullName, Email, PasswordHash, Role, CreatedAt, LastLogin
Purpose: Authentication, access control, and activity tracking.
2. Patient: Stores demographic and contact details of each patient.
Attributes:
PatientID, FirstName, LastName, DOB, Gender, Email, Phone, Address, MedicalRecordNo, CreatedAt
Purpose: Acts as the core entity linking recordings, transcriptions, and reports.
3. Recording: Stores metadata of each voice memo.
Attributes:
RecordingID, PatientID (FK), UserID (FK), FilePath, Duration, RecordingDate, Status
Purpose: Tracks all recorded sessions between clinician and patient.
4. Transcription: Stores the text version of the recorded audio.
Attributes:
TranscriptionID, RecordingID (FK), TranscriptionText, AccuracyScore, CreatedAt
Purpose: Converts unstructured audio into structured text linked to a recording.
5. Report: Stores the finalized formatted notes (Progress or Nurse notes).
Attributes:
ReportID, TranscriptionID (FK), PatientID (FK), UserID (FK), ReportType, ReportText, CreatedAt, Status
Purpose: Represents the main output of the workflow — the clinical report.
6. Template: Stores report templates used for formatting notes.
Attributes:
TemplateID, ReportType, TemplateName, TemplateText, Specialty, LastUpdated
Purpose: Allows doctors to choose or modify structured templates for reports.

7. AuditLog: Tracks all user activity in the system.
Attributes:
LogID, UserID (FK), ActionType, EntityName, EntityID, OldValue, NewValue, ActionTimestamp
Purpose: Supports compliance, accountability, and recovery.
8. EmailLog: Tracks report delivery to patients.
Attributes:
EmailID, ReportID (FK), PatientID (FK), RecipientEmail, SentAt, Status
Purpose: Records when and how reports are shared securely.

2. Relationships Between Entities

Relationship	Type	Description
User → Recording	1-to-Many	Each user (doctor) can create many recordings.
Patient → Recording	1-to-Many	Each patient may have multiple recordings linked to their ID.
Recording → Transcription	1-to-1	Every recording has one transcription.
Transcription → Report	1-to-1	Each transcription produces one report.
Patient → Report	1-to-Many	Patients can have multiple reports over time.
User → Report	1-to-Many	Each doctor can generate many reports.
Template → Report	1-to-Many	Reports can be created using different templates.
User → AuditLog	1-to-Many	Tracks each user's actions.
Report → EmailLog	1-to-Many	Each report may be sent to a patient multiple time.

3. Constraints

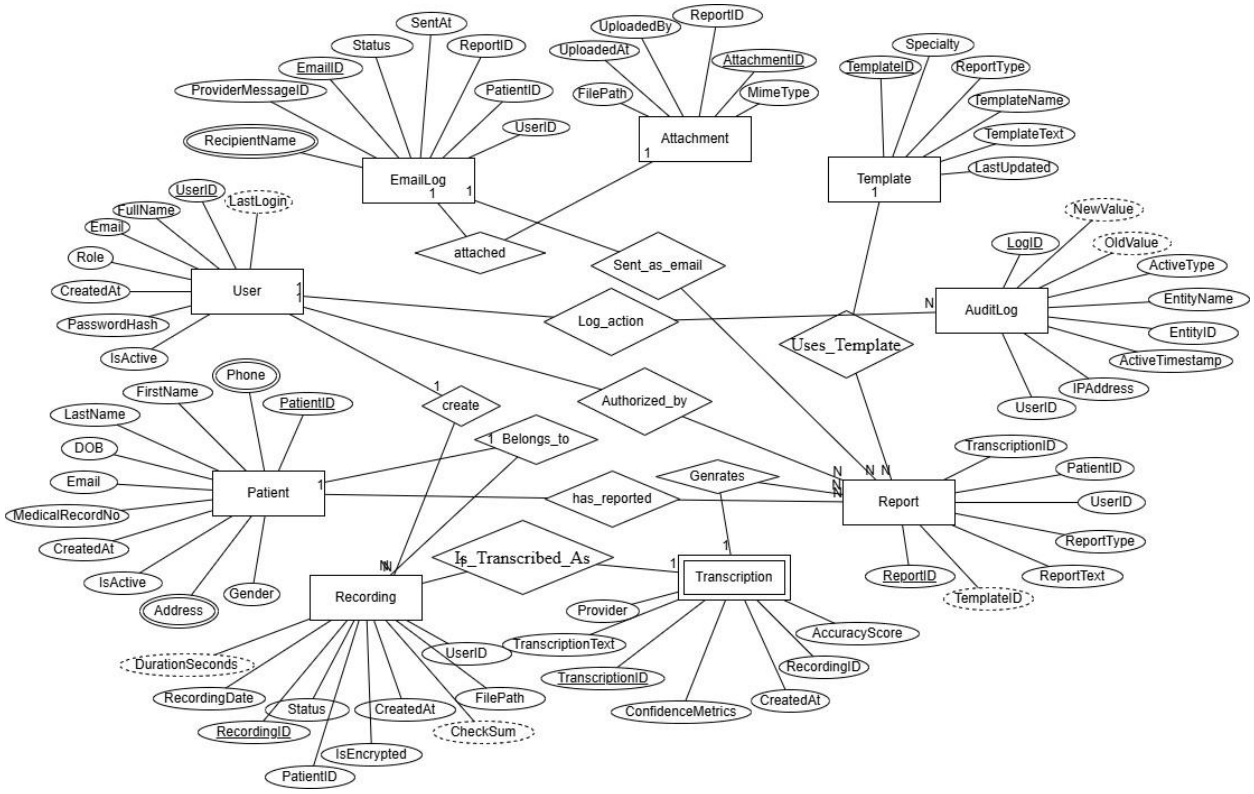
Constraint Type	Description
Primary Key (PK)	Uniquely identify each record in a table. (e.g., PatientID, RecordingID)
Foreign Key (FK)	Maintains referential integrity (e.g., Recording.PatientID → Patient.PatientID).
Unique Constraint	Ensures unique Email for Users, unique MedicalRecordNo for Patients.
Not Null Constraint	Important fields (e.g., names, file paths, timestamps) cannot be null.
Check Constraint	Validates values (e.g., ReportType ∈ {'Progress', 'Nurse'}).
Default Constraint	Auto-sets CreatedAt with current timestamp.
Referential Integrity	Prevents deletion of parent records if children exist (ON DELETE RESTRICT).
Access Control Constraint	Only Admins can delete; Doctors can insert/update.
Audit Logging Constraint	Every INSERT, UPDATE, DELETE action generates an AuditLog entry.
Data Security Constraint	Encrypt patient contact data and file paths for recordings.

2. CONCEPTUAL & LOGICAL DESIGN:

The conceptual design of the Relivox system focuses on modeling the core clinical workflow by identifying the major entities involved in capturing and managing medical voice memos. The system is built around key entities such as **User**, **Patient**, **Recording**, **Transcription**, **Report**, **Template**, **EmailLog**, and **AuditLog**, each representing essential components of the documentation process. Users and patients form the foundation of the model, while recordings capture voice notes that are later converted into structured transcriptions. These transcriptions support the generation of medical reports, which may optionally use predefined templates for standardized formatting.

The relationships between entities reflect real-world healthcare interactions. A user creates many recordings, each linked to a specific patient; every recording produces one transcription, and each transcription may generate one or more reports. Reports can be emailed, with each delivery tracked in the EmailLog, while all system actions are monitored through the AuditLog to ensure compliance and accountability. Together, these entities and relationships provide a complete conceptual structure for supporting the full lifecycle of clinical documentation within the Relivox platform.

2.1 ER Diagram:



The ER diagram illustrates the core structure of the Relivox Medical Voice Memo & Patient Record System by showing how all major entities interact within the clinical documentation workflow. At the center of the model, User, Patient, Recording, Transcription, and Report form the primary data flow. A User (doctor or clinician) creates multiple Recordings, each linked to a single Patient. Every recording is then converted into exactly one Transcription, which can generate one or more Reports, reflecting different types of clinical documentation such as progress notes or nurse notes.

Supporting entities enhance system functionality. Template provides predefined report formats that can be reused across many reports, while Attachment stores files linked to reports such as images or PDFs. EmailLog records when reports are emailed to patients, ensuring traceability of communication. AuditLog tracks important user actions such as creation, updates, or status changes to maintain transparency, accountability, and compliance with healthcare documentation standards.

Overall, the ER diagram captures a complete, interconnected data structure where each entity has a clear role, and all relationships reflect real-world interactions within the clinical workflow. This model ensures data integrity, accurate tracing of clinical events, and efficient retrieval of patient information throughout the Relivox system.

2.2 Normalization:

The normalization process ensures that the Relivox database structure eliminates redundancy, maintains data integrity, and supports efficient query performance. Starting from the initial relational schema generated through ER-to-relational mapping, each table was evaluated against the rules of First, Second, and Third Normal Form.

2.2.1 First Normal Form (1NF)

In First Normal Form (1NF), all attributes must contain atomic values with no repeating groups. The Relivox schema satisfies this requirement because all fields use atomic data types such as strings, numbers, and timestamps. Although the ConfidenceMetrics attribute in the Transcription table is stored as JSON, it is treated as a single atomic value for the purposes of this design, with options to further normalize it into a separate table if needed.

2.2.2 Second Normal Form (2NF)

In Second Normal Form (2NF), every non-key attribute must depend on the entire primary key. Since all tables in the Relivox system use single-column surrogate primary keys, partial

dependencies are not possible. The schema does not include composite primary keys, and therefore 2NF is fully satisfied without requiring additional restructuring.

2.2.3 Third Normal Form (3NF)

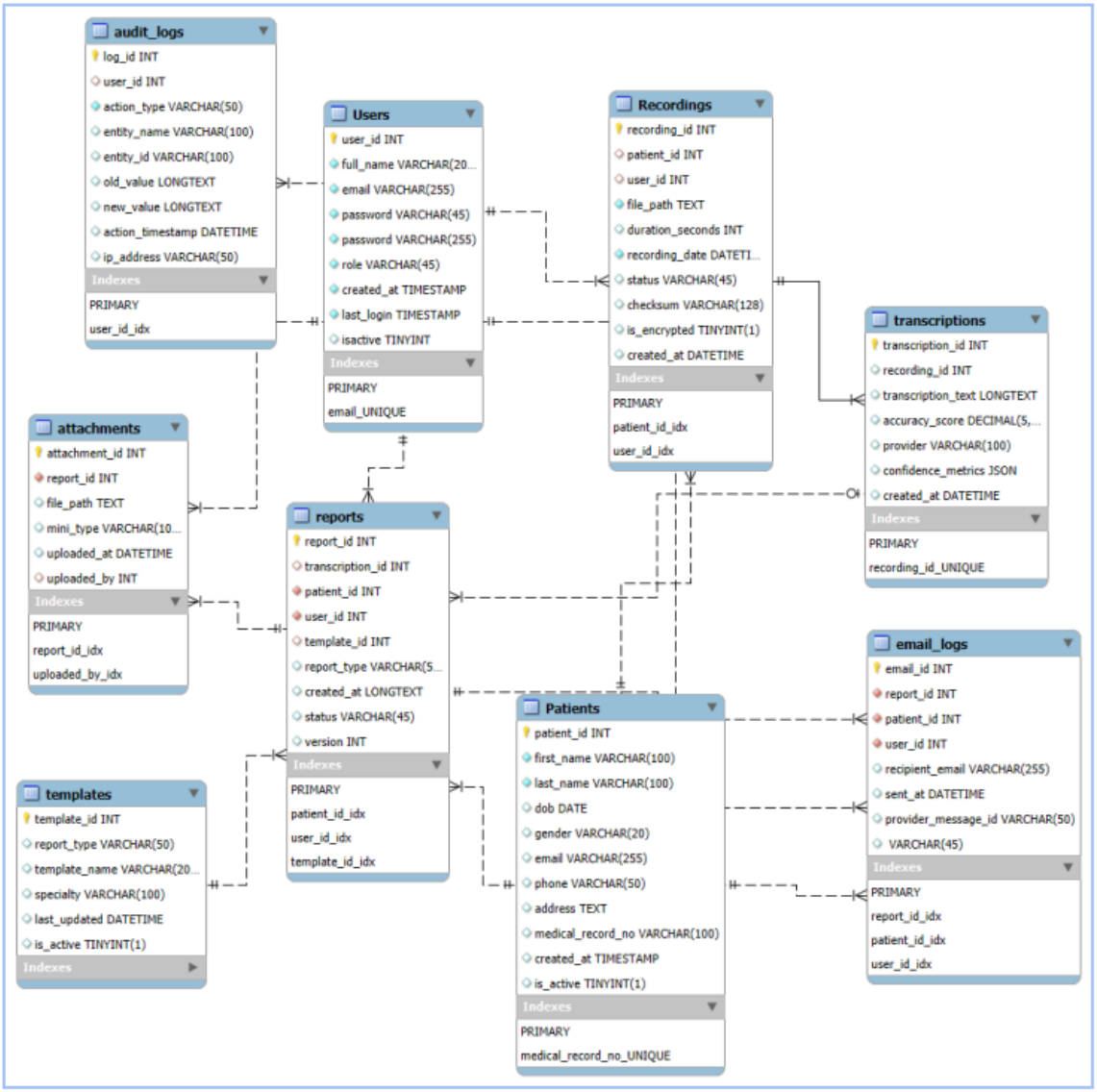
In Third Normal Form (3NF), there must be no transitive dependencies, meaning non-key attributes cannot depend on other non-key attributes. The Patient table, for example, stores demographic attributes that depend solely on PatientID, and unique identifiers like MedicalRecordNo introduce no transitive dependencies. Similarly, the Report table keeps its report details separate from Template information, avoiding duplication. Transcription attributes depend entirely on TranscriptionID, and the AuditLog stores independent snapshot values without any transitive dependencies. Overall, the scheme meets all 3NF requirements, resulting in a well-organized and efficient database structure suitable for the system's operational needs.

2.3EER Diagram

The ER diagram models the Relivox database as an interconnected set of relational entities supporting clinical voice-to-text documentation. The **Users** table maintains system credentials, role classifications, and activity metadata, and establishes a one-to-many relationship with both **Recordings** and **Reports**, indicating which clinician generated each artifact. **Patients** store demographic and medical record identifiers, with mandatory foreign key relationships linking each recording, transcription, and report back to a specific patient record.

The **Recordings** entity captures raw audio events with attributes for file storage paths, encryption state, duration, checksum validation, and timestamps. Each recording maintains a one-to-one relationship with the **Transcriptions** table, where transcription output, accuracy scoring, and confidence metrics have persisted. Transcriptions serve as the source for the **Reports** entity, where multiple clinical reports can be derived from a single transcription. Reports maintain foreign key references to **Users**, **Patients**, **Transcriptions**, and optionally **Templates**, which define reusable formatting structures and standardized medical text blocks.

Auxiliary entities support additional system functionality. **Attachments** link external files to corresponding reports, enforcing referential integrity via report-level foreign keys. **EmailLog** records outbound communication events, capturing the destination email, provider message body, and the initiating user, while ensuring traceability of report dissemination. The **AuditLog** table captures all system-level mutation events by storing entity identifiers, old and new values, action timestamps, and originating IP addresses, enabling complete reconstruction of user actions for security and compliance.


audit_logs
log_id INT
user_id INT
action_type VARCHAR(50)
entity_name VARCHAR(100)
entity_id VARCHAR(100)
old_value LONGTEXT
new_value LONGTEXT
action_timestamp DATETIME
ip_address VARCHAR(50)
Indexes
PRIMARY
user_id_idx
Users
user_id INT
full_name VARCHAR(20...
email VARCHAR(255)
password VARCHAR(45)
password VARCHAR(255)
role VARCHAR(45)
created_at TIMESTAMP
last_login TIMESTAMP
isactive TINYINT
Indexes
PRIMARY
email_UNIQUE
Recordings
recording_id INT
patient_id INT
user_id INT
file_path TEXT
duration_seconds INT
recording_date DATETI...
status VARCHAR(45)
checksum VARCHAR(128)
is_encrypted TINYINT(1)
created_at DATETIME
Indexes
PRIMARY
patient_id_idx
user_id_idx
transcriptions
transcription_id INT
recording_id INT
transcription_text LONGTEXT
accuracy_score DECIMAL(5,...
provider VARCHAR(100)
confidence_metrics JSON
created_at DATETIME
Indexes
PRIMARY
recording_id_UNIQUE
attachments
attachment_id INT
report_id INT
file_path TEXT
mini_type VARCHAR(10...
uploaded_at DATETIME
uploaded_by INT
Indexes
PRIMARY
report_id_idx
uploaded_by_idx
reports
report_id INT
transcription_id INT
patient_id INT
user_id INT
template_id INT
report_type VARCHAR(5...
created_at LONGTEXT
status VARCHAR(45)
version INT
Indexes
PRIMARY
patient_id_idx
user_id_idx
template_id_idx
email_logs
email_id INT
report_id INT
patient_id INT
user_id INT
recipient_email VARCHAR(255)
sent_at DATETIME
provider_message_id VARCHAR(50)
VARCHAR(45)
Indexes
PRIMARY
report_id_idx
patient_id_idx
user_id_idx
templates
template_id INT
report_type VARCHAR(50)
template_name VARCHAR(20...
specialty VARCHAR(100)
last_updated DATETIME
is_active TINYINT(1)
Indexes
Patients
patient_id INT
first_name VARCHAR(100)
last_name VARCHAR(100)
dob DATE
gender VARCHAR(20)
email VARCHAR(255)
phone VARCHAR(50)
address TEXT
medical_record_no VARCHAR(100)
created_at TIMESTAMP
is_active TINYINT(1)
Indexes
PRIMARY
medical_record_no_UNIQUE

(EER DIAGRAM)

3. PHYSICAL DESIGN & PERFORMANCE

The physical design phase translates the logical schema of the Relivox system into an operational MySQL database optimized for accuracy, reliability, and clinical performance requirements. Using MySQL Workbench, each table was implemented with appropriate data types, surrogate primary keys, and foreign key constraints to enforce the relationships defined during conceptual and logical modeling. INT AUTO_INCREMENT identifiers were used across all core entities including Users, Patients, Recordings, Transcriptions, and Reports to simplify key management and maintain consistent referential integrity. Constraints such as NOT NULL, UNIQUE, and CHECK expressions were applied where appropriate to ensure that essential clinical attributes such as medical record numbers, timestamps, file paths, and transcription details remain valid and consistent during data entry.

During implementation, additional physical-level considerations were applied to support the workflow of the Relivox system. Attributes were assigned precise data types to optimize storage and retrieval performance, such as using VARCHAR for short text fields, LONGTEXT for transcription and report bodies, DECIMAL for accuracy metrics, and JSON for confidence metrics. Each table was assigned foreign key references to enforce hierarchical dependencies, for example, ensuring that a report cannot exist without its corresponding transcription or patience. Cascading options such as ON DELETE RESTRICT were applied to prevent accidental removal of clinical data with dependent entities, a critical requirement for healthcare systems that must maintain auditable, long-term records. To support system functionality beyond simple CRUD operations, physical design incorporates several advanced SQL components. Views were created to simplify frequent multi-table queries, such as retrieving all reports for a given patient or identifying pending emails. Stored procedures and functions were implemented to automate repetitive system behavior, including timestamp generation, audit logging, and report creation tasks that must remain consistent across multiple application modules. System triggers were added to monitor and record changes to high-value tables such as Patients, Reports, and Transcriptions. For example, updates to patient demographics or report text generate corresponding entries in the AuditLog table, preserving a complete history of modifications for compliance and diagnostics.

Performance optimization formed a critical part of the physical design process. Indexes were applied to columns frequently used in filters, joins, and sorting operations, including PatientID, UserID, RecordingDate, ReportType, and Email fields—to reduce query response times for high-traffic operations. Query plans were examined in MySQL Workbench to ensure efficient table scans and join operations, particularly in complex retrieval tasks where multiple relationships intersect. Sample data was inserted into the system to validate schema behavior, test constraints, and analyze performance under simulated clinical workloads. The database structure was evaluated for scalable operation, enabling it to handle increasing volumes of recordings, transcriptions, and reports as the system expands.Together, the implemented tables, constraints, triggers, and indexing strategies ensure that the Relivox database operates efficiently under real-world clinical usage. The physical design supports the high frequency of inserts (recordings, transcriptions, log events), frequent retrieval operations (patient records, historical reports), and periodic update operations while maintaining integrity, reliability, and consistent performance across all system components.

4. FRONTEND DESIGN & IMPLEMENTATION

The Relivox system is a single-page PHP web application designed to interact seamlessly with a MySQL database for managing medical users and patient records. This project demonstrates a fully functional database-driven application that implements core CRUD (Create, Read, Update, Delete) operations while also showcasing advanced database concepts such as Views, Functions, Stored Procedures, and Triggers. The system was developed as part of the Advanced Database Systems course, with deployment on Rowan University's Elvis web server, ensuring accessibility and compliance with the course requirements.

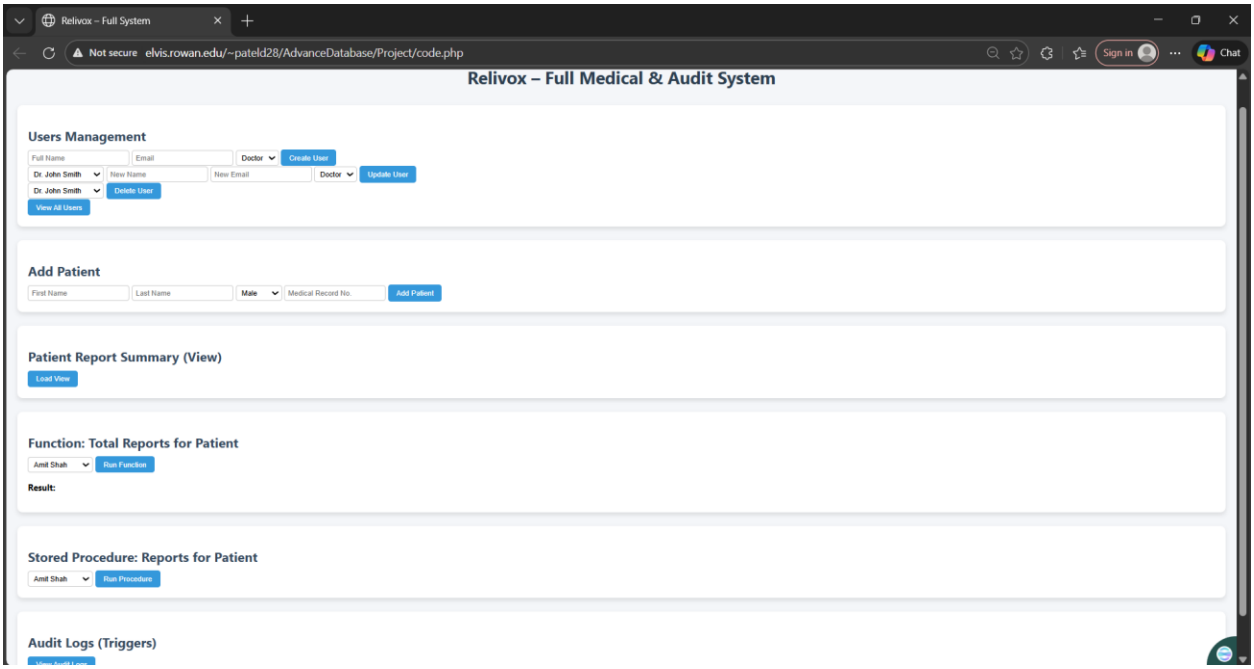
The primary goal of the application is to provide a centralized interface for managing medical users including doctors, nurses, administrators and patients. The system allows for the creation of new records, modification of existing records, and deletion of records as necessary. All operations are performed directly on the MySQL database through PHP server-side scripts, ensuring that changes persist in real time. Users are displayed in a tabular format, making it easy to browse, update, or remove individual records, while patients are similarly managed with dedicated forms for inserting new patient details.

Beyond basic CRUD operations, the system demonstrates the use of database Views, which provide a summarized perspective of patient reports through a dedicated “Patient Report Summary” view. Functions are used to perform calculations on the database, such as determining the total number of reports associated with a specific patient. Stored Procedures provide a structured way to retrieve detailed report information for each patient, encapsulating complex SQL queries into reusable database routines. Triggers are implemented to maintain an audit trail, automatically logging all insertions and updates to the audit_logs table, thereby providing a mechanism for monitoring and ensuring data integrity.

The application’s user interface is designed to be intuitive, organized into clearly defined sections for Users, Patients, Views, Functions, Stored Procedures, and Audit Logs. Each section is visually distinct using card-style layouts, and interactive forms enable users to perform operations without navigating to multiple pages. Tables are formatted for readability, with alternating row colors and descriptive headers, ensuring that the information is easy to interpret.

Overall, the Relivox system successfully integrates both frontend and backend elements to create a robust, single-page web application that demonstrates practical database management skills. It highlights the developer’s ability to implement a wide range of database operations while maintaining a clean, user-friendly interface. This project serves as a comprehensive showcase of advanced MySQL and PHP integration, providing a fully functional prototype that is accessible, efficient, and easily extendable for future enhancements.

Link of the webpage: http://elvis.rowan.edu/~pateld28/AdvanceDatabase/Project/code.php



The frontend implementation of the Relivox system provides an accessible, browser-based interface that enables clinicians and administrators to interact directly with the underlying MySQL database. Developed primarily using PHP, HTML, CSS, and MySQLi queries, the interface acts as an application layer connecting users to backend operations such as creating patients, generating reports, updating user accounts, and retrieving audit logs. The design follows a modular, section-based layout where each functional block corresponds to a database operation or stored routine.

The Users Management module allows administrators to create, update, and delete system users while assigning role types such as “Doctor” or “Administrator.” This module interacts directly with the Users table and ensures that each update triggers the corresponding audit log entry. The Add Patient section provides clinicians with a simple form to register new patients by capturing first name, last name, gender, and medical record number. Through backend PHP logic, each patient insertion is validated and saved to the Patients table with referential integrity enforced by database constraints.

Additional sections of the interface demonstrate the system’s use of SQL views, stored functions, and stored procedures. For example, the Patient Report Summary (View) button retrieves and displays data stored in a database view that joins patient, transcription, and report information. Similarly, the Function: Total Reports for Patient module calls a stored function that returns the number of reports associated with a specific patient. The Stored Procedure: Reports for Patient section executes a predefined SQL procedure that retrieves structured report details using input parameters selected from the dropdown menu.

Finally, the Audit Logs (Triggers) module showcases system activity captured through database-level triggers. Every insertion, update, or modification of sensitive tables—such as Users, Patients, and Reports—automatically generates an entry in the AuditLog table. The frontend retrieves and displays these logs, providing full traceability of user actions and supporting compliance with medical documentation standards.

Together, these implementation components demonstrate how the Relivox frontend integrates seamlessly with the backend database, providing a functional, secure, and user-friendly interface for managing clinical data and validating physical design through live operations.

5. CONCLUSION:

The Relivox Medical Voice Memo & Patient Record System brings together the full lifecycle of database development, beginning with understanding clinical workflow requirements and ending with a functional, deployed system. Through careful analysis of how clinicians capture and manage patient information, the project established a clear data model that accurately reflects the relationships among users, patients, recordings, transcriptions, and reports. Each stage of design from conceptual to logical to physical ensured that the structure of the database remained consistent, secure, and optimized for medical use. Normalization eliminated redundancy, while referential integrity and carefully chosen constraints maintained the accuracy and reliability required in healthcare environments.

The implementation further demonstrated the system's practicality. By using MySQL Workbench, PHP, and a browser-based interface, the project delivered an application capable of performing real operations such as adding users, registering patients, generating reports, and viewing audit logs. The use of triggers, functions, stored procedures, and views validated the backend design and showcased how automation and performance considerations were integrated into the system. With its clean interface and strong database foundation, Relivox successfully simulates a clinical documentation platform that can scale and evolve with additional features such as transcription APIs, template expansion, and integration with electronic health record systems.